VIP-шпаргалка: Трансформери та великі мовні моделі

Афшин Аміді та Шервін Аміді
Переклад українською: Євген Шморгун

13 вересня 2026 р.

Ця VIP-шпаргалка дає огляд змісту книги "Super Study Guide: Transformers & Large Language Models", яка містить ∼600 ілюстрацій на 250 сторінках і детально розглядає наведені нижче концепції. Більше інформації: `https://superstudy.guide`.

1 Основи

1.1 Токени

❒ **Визначення** – *Токен* — це неподільна одиниця тексту, як-от слово, підслово чи символ, що належить до заздалегідь визначеного словника.

Примітка: невідомий токен `[UNK]` *позначає невідомі фрагменти тексту, а токен доповнення* `[PAD]` *заповнює порожні позиції, щоб вирівняти довжину вхідних послідовностей.*

❒ **Токенізатор** – *Токенізатор* T розбиває текст на токени довільного рівня деталізації.

this teddy bear is reaaaally cute → T → `this` `teddy bear` `is` `[UNK]` `cute` `[PAD]` ... `[PAD]`

Ось основні типи токенізаторів:

Тип	Переваги	Недоліки	Ілюстрація
Слово	• Легко інтерпретувати • Коротка послідовність	• Великий словник • Словоформи не враховуються	`teddy` `bear`
Підслово	• Враховує корені слів • Інтуїтивні ембедінги	• Довша послідовність • Складніша токенізація	`ted` `##dy` `bear`
Символ Байт	• Немає проблеми слів поза словником • Малий словник	• Значно довша послідовність • Патерни важко тлумачити через надто дрібні одиниці	`t` `e` `d` `d` `y` ␣ `b` `e` `a` `r`

Примітка: BPE (Byte-Pair Encoding) та Unigram — поширені токенізатори рівня підслів.

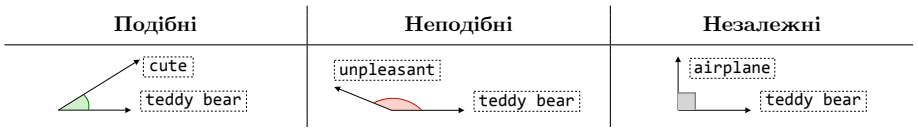
1.2 Ембедінги

❒ **Визначення** – *Ембедінг* — це числове представлення елемента (наприклад, токена чи речення), що задається вектором $x \in \mathbb{R}^n$.

❒ **Подібність** – *Косинусна подібність* між двома токенами t_1, t_2 обчислюється так:

$$\text{similarity}(t_1, t_2) = \frac{t_1 \cdot t_2}{||t_1||\ ||t_2||} = \cos(\theta) \in [-1, 1]$$

Кут θ характеризує подібність між двома токенами:

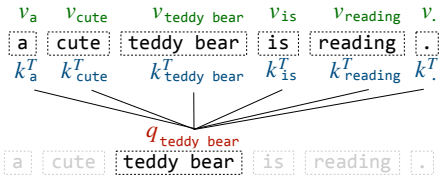


Примітка: наближений пошук сусідів (Approximate Nearest Neighbors, ANN) і локально-чутливе хешування (Locality Sensitive Hashing, LSH) ефективно апроксимують обчислення подібності у великих базах даних.

2 Трансформери

2.1 Увага

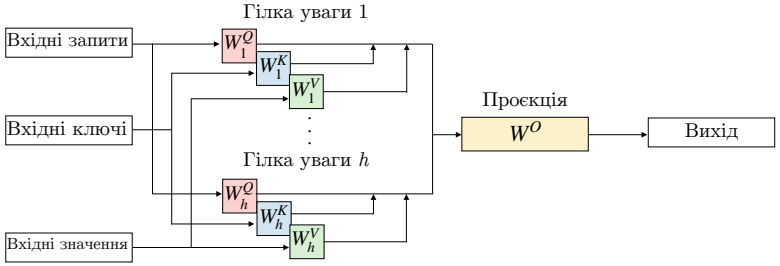
❒ **Формула** – Маючи *запит* q, ми хочемо знати, на який *ключ* k запит має звернути «увагу» (attention) з огляду на відповідне *значення* v.



Увагу можна ефективно обчислити за допомогою матриць Q, K, V, що містять запити q, ключі k та значення v відповідно, а також розмірності ключів d_k:

$$\text{attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

❒ **MHA** – Шар *багатоголової уваги* (*Multi-Head Attention*, MHA) обчислює увагу паралельно в кількох *гілках* (heads), а потім проєктує результат у вихідний простір.

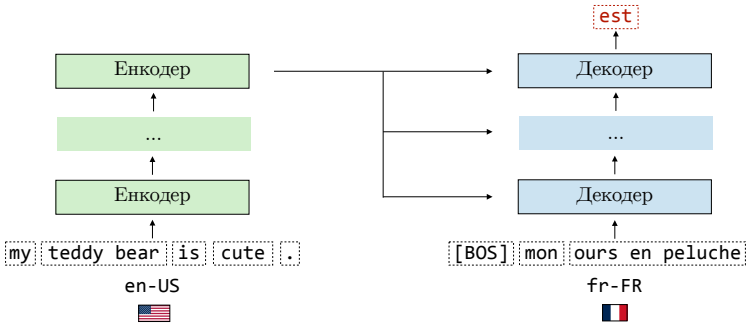


Він складається з h гілок уваги, а також матриць W^Q, W^K, W^V, які проєктують вхідні дані, щоб отримати запити Q, ключі K та значення V. Проєкція виконується за допомогою матриці W^O.

Примітка: увага з групованими запитами (Grouped-Query Attention, GQA) і багатозапитова увага (Multi-Query Attention, MQA) — варіанти MHA, які зменшують обчислювальні витрати завдяки спільним для кількох гілок ключам і значенням.

2.2 Архітектура

❒ **Огляд** – *Трансформер* — модель, що ґрунтується на механізмі самоуваги та складається з енкодерів і декодерів. Енкодери обчислюють змістовні ембедінги вхідних даних, які потім використовують декодери, щоб передбачити наступний токен у послідовності.

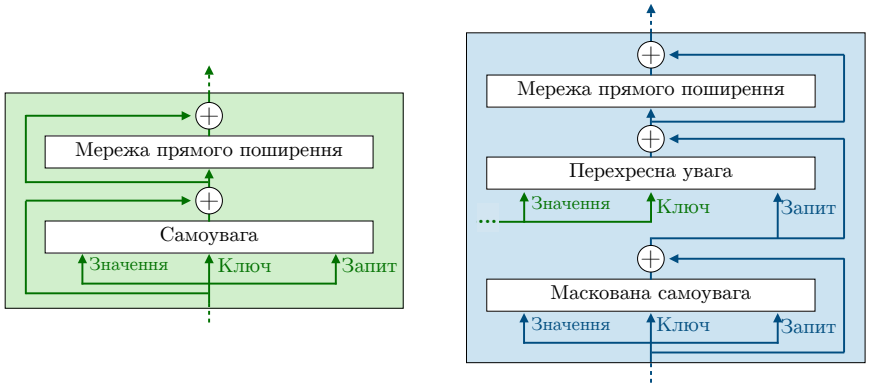


Примітка: хоча Трансформер спочатку запропонували як модель для задач перекладу, нині його широко застосовують і в багатьох інших сферах.

❒ **Компоненти** – *Енкодер* і *декодер* — два основні компоненти Трансформера, що виконують різні ролі:

Енкодер
Закодовані ембедінги містять зміст вхідних даних

Декодер
Декодовані ембедінги містять зміст і вхідних даних, і вже передбаченого виходу

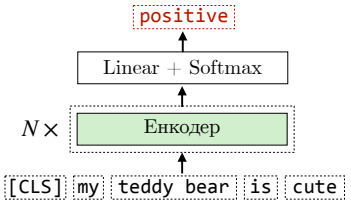


❒ **Позиційні ембедінги** – *Позиційні ембедінги* вказують, де саме в послідовності розташований токен. Їх можна або задати наперед, або навчити на даних.

Примітка: ротаційні позиційні ембедінги (Rotary Position Embeddings, RoPE) — популярний та ефективний варіант, що обертає вектори запитів і ключів, щоб урахувати відносні позиції токенів.

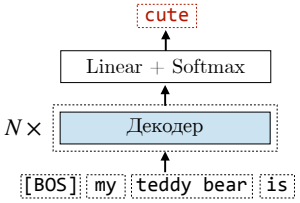
2.3 Варіанти

❒ **Лише енкодер** – *BERT* (*Bidirectional Encoder Representations from Transformers*) — модель на основі Трансформера, що складається зі стека енкодерів, приймає на вхід текст і видає змістовні ембедінги, які згодом можна використати в прикладних задачах класифікації.



На початок послідовності додають токен [CLS], який вбирає зміст речення. Його закодований ембедінг часто використовують у прикладних задачах, як-от аналіз тональності.

❒ **Лише декодер** – *GPT* (*Generative Pre-trained Transformer*) — авторегресійна модель на основі Трансформера, що складається зі стека декодерів. На відміну від BERT і похідних від неї моделей, GPT розглядає всі задачі як задачі «текст у текст».



Більшість сучасних передових LLM, як-от серія GPT, DeepSeek, Mistral, Gemma тощо, мають архітектуру лише з декодером.

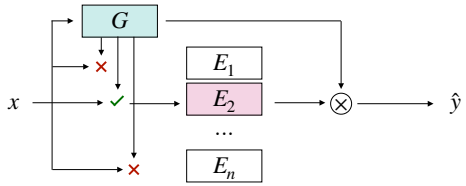
Примітка: моделі «енкодер-декодер», як-от T5, теж є авторегресійними та мають багато спільних рис із моделями лише з декодером.

3 Великі мовні моделі

3.1 Огляд

❒ **Визначення** – *Велика мовна модель* (*Large Language Model*, LLM) — модель на основі Трансформера з потужними можливостями обробки природної мови (NLP). Вона «велика» в тому сенсі, що зазвичай містить мільярди параметрів і навчається на великому обсязі даних.

❒ **Суміш експертів** – *Суміш експертів* (*Mixture of Experts*, MoE) — модель, яка під час інференсу активує лише частину нейронів. Вона ґрунтується на маршрутизаторі (gate) G та експертах E_1, ..., E_n.



$$\hat{y} = \sum_{i=1}^{n} G(x)_i E_i(x)$$

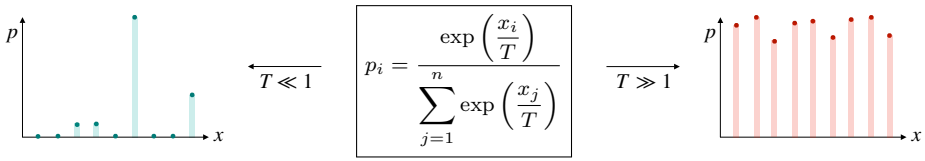
LLM на основі MoE застосовують такий маршрутизатор у своїх FFNN: для кожного токена він обчислює ваги експертів і спрямовує токен лише до кількох із них.

3.2 Промптинг

❒ **Довжина контексту** – *Довжина контексту* моделі — це максимальна кількість токенів, що вміщується у вхідні дані. Зазвичай вона становить від десятків тисяч до мільйонів токенів.

Примітка: якість відповідей LLM зазвичай погіршується в міру заповнення контекстного вікна; це явище відоме як «гниття контексту» (context rot).

❒ **Семплювання при декодуванні** – Токени семплюються з передбаченого розподілу ймовірностей p_i, яким керує гіперпараметр — температура T.



Примітка: висока температура призводить до креативніших результатів, а низька — до більш детермінованих.

4 Навчання LLM

4.1 Попереднє навчання

❒ **Втрати перехресної ентропії** – *Перехресна ентропія* (*Cross-Entropy*, CE) — функція втрат, що штрафує модель, коли та призначає низьку ймовірність p_i правильному наступному токену i:

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{V} y_i \log p_i$$

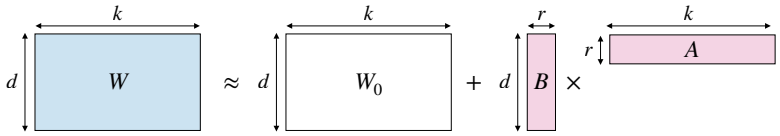
❒ **Закони масштабування** – *Закони масштабування* — емпіричні закономірності передбачуваного степеневого зростання якості моделі зі збільшенням її розміру, навчальних даних та обчислень.

4.2 Донавчання

❒ **SFT** – *Донавчання з учителем* (*Supervised Fine-Tuning*, SFT) — підхід пост-навчання, що узгоджує поведінку моделі з кінцевою задачею. Він спирається на якісні пари «вхід — вихід» для цієї задачі.

Примітка: якщо дані SFT — це інструкції, цей крок називають налаштуванням на інструкції.

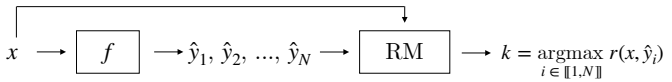
❒ **PEFT** – *Параметрично ефективне донавчання* (*Parameter-Efficient Fine-Tuning*, PEFT) — категорія методів для ефективного виконання SFT. Зокрема, *низькорангова адаптація* (*Low-Rank Adaptation*, LoRA) апроксимує навчувані ваги W, фіксуючи W_0 й натомість навчаючи низькорангові матриці A, B:



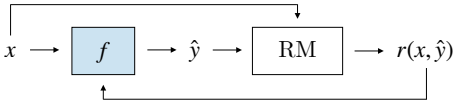
Примітка: інші методи PEFT — префіксне налаштування та вставлення шарів-адаптерів.

4.3 Налаштування за вподобаннями

❒ **Модель винагороди** – *Модель винагороди* (*Reward Model*, RM) передбачає, наскільки добре вихід $\hat{y}$ відповідає бажаній поведінці для входу x. *Best-of-N* (BoN), або *вибірка з відхиленням* (rejection sampling), — метод, що за допомогою RM обирає найкращу з N згенерованих відповідей.



❒ **Навчання з підкріпленням** – *Навчання з підкріпленням* (*Reinforcement Learning*, RL) — підхід, що використовує RM та оновлює модель f на основі винагород за її виходи. Якщо RM ґрунтується на вподобаннях людей, процес називають *навчанням з підкріпленням на основі відгуків людей* (*Reinforcement Learning from Human Feedback*, RLHF).

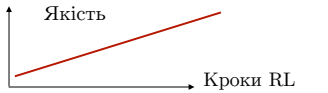
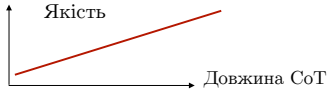


Примітка: існують і підходи з учителем, як-от пряма оптимізація вподобань (Direct Preference Optimization, DPO), що поєднують RM та RL в одному кроці навчання з учителем.

4.4 Міркування

❒ **Ланцюжок міркувань** – *Ланцюжок міркувань* (*Chain-of-Thought*, CoT) — процес, під час якого модель розбиває складну задачу на низку проміжних кроків. Це допомагає моделі згенерувати правильну остаточну відповідь.

❒ **Модель міркувань** – *Модель міркувань* — модель, що спирається на ланцюжки міркувань у стилі CoT, щоб розв'язувати складніші задачі з математики, програмування та логіки. Для посилення здатності міркувати використовують два типи масштабування:

	Опис	**Ілюстрація**
Масштабування при навчанні	Довше навчати з RL, щоб модель навчилася генерувати ланцюжки міркувань CoT перед відповіддю	
Масштабування при інференсі	Дати моделі довше подумати перед відповіддю через ключові слова budget forcing, як-от `"Wait"`	

❒ **OPD** – *Дистиляція на власній політиці* (*On-Policy Distillation*, OPD) — підхід пост-навчання, що спирається на траєкторії, згенеровані самою моделлю, і використовує сильнішу модель-учителя для нагляду на рівні токенів на цих траєкторіях.

Примітка: у класичному RL модель отримує одну винагороду за всю згенеровану послідовність — це розріджений сигнал. OPD натомість дає щільний сигнал: учитель оцінює кожен токен уздовж усієї згенерованої траєкторії.

5 Навчання з підкріпленням для LLM

5.1 Основи

❒ **Постановка** – У застосуваннях RL до LLM:

- *Агент* — це мовна модель, чия *політика* задає розподіл імовірностей над діями — наступними токенами.
- *Стан* — це промпт разом з уже згенерованими токенами.
- *Дія* — це наступний семплований токен.
- *Винагорода* оцінює згенеровану відповідь відповідно до бажаної поведінки чи результату задачі.

❒ **Градієнти політики** – *Градієнт політики* — сімейство алгоритмів RL, які оновлюють політику, надаючи перевагу діям із вищою семплованою віддачею R. Оскільки R може суттєво різнитися між траєкторіями, можна використати базову лінію (baseline) $b(s)$, щоб оцінити, наскільки дії кращі за очікуване: $R - b(s)$. Це допомагає зменшити дисперсію оновлення.

Примітка: REINFORCE — приклад алгоритму градієнта політики.

5.2 PPO

❒ **Функція цінності** – *Функція цінності* оцінює очікувану майбутню винагороду з певного стану чи пари «стан — дія» за поточної політики.

❒ **Алгоритм** – *Проксимальна оптимізація політики* (*Proximal Policy Optimization*, PPO) — алгоритм RL, що максимізує винагороди, утримуючи модель біля базової, щоб запобігти зламу винагороди. Задавши відношення політик $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$, маємо:

$$\boxed{\mathcal{L}_{\text{PPO}} = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right]}$$

5.3 GRPO

❒ **Перевірювані винагороди** – *Перевірювані винагороди* обчислюються автоматично на основі об'єктивно перевірюваних результатів, як-от точні відповіді, виконання коду чи формальні доведення.

❒ **Алгоритм** – *Групова відносна оптимізація політики* (*Group Relative Policy Optimization*, GRPO) оцінює переваги $\hat{A}_i$ через порівняння винагород R_i у групі з G відповідей, що усуває потребу в окремій моделі цінності. Задавши $\hat{A}_i = \frac{R_i - \text{mean}(R_1,\ldots,R_G)}{\text{std}(R_1,\ldots,R_G)}$, маємо:

$$\boxed{\mathcal{L}_{\text{GRPO}} = \frac{1}{G}\sum_{i=1}^{G}\mathbb{E}_t\left[\min\left(r_{i,t}(\theta)\hat{A}_i,\ \text{clip}\left(r_{i,t}(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_i\right) - \beta D_{\text{KL}}\left(\pi_\theta \| \pi_{\text{ref}}\right)\right]}$$

Примітка: існують розширення, що усувають обмеження GRPO. Наприклад, Dr. GRPO виправляє зміщення, пов'язане з довжиною, а DAPO покращує стабільність навчання та дослідження завдяки модифікованому обрізанню (clipping) і динамічному семплюванню.

6 Системи LLM

6.1 Ядра (kernels)

❒ **Flash attention** – *Flash attention* — точний метод, що оптимізує обчислення уваги, вміло використовуючи GPU: матричні операції виконуються у швидкій статичній пам'яті (SRAM), а результати потім записуються в повільнішу пам'ять з високою пропускною здатністю (HBM).

Примітка: на практиці це зменшує використання пам'яті та пришвидшує обчислення.

❒ **Paged attention** – *Paged attention* керує KV-кешем у блоках пам'яті фіксованого розміру, що зменшує фрагментацію й дає змогу ефективно ділити пам'ять між послідовностями.

6.2 Розподілене навчання

❒ **Паралелізм** – *Паралелізм* розподіляє навчання LLM між кількома пристроями, розбиваючи дані, обчислення моделі чи її компоненти.

❒ **Типи** – Основні форми паралелізму:

- *Паралелізм даних*: модель дублюється, а навчальні батчі розподіляються між пристроями.
- *Тензорний паралелізм*: обчислення в межах шарів розподіляються між пристроями.
- *Конвеєрний паралелізм*: шари моделі поділено на етапи на різних пристроях.
- *Експертний паралелізм*: експерти MoE на різних пристроях, токени маршрутизуються до них.

Примітка: масштабне навчання LLM зазвичай поєднує кілька форм паралелізму.

6.3 Оптимізації інференсу

❒ **KV-кешування** – *Кешування ключів і значень* (*Key-Value caching*, KV) зберігає раніше обчислені під час декодування вектори ключів і значень, усуваючи зайві обчислення уваги для кожного нового токена.

Примітка: кешування промптів повторно використовує KV-кеш для однакових префіксів.

❒ **Prefill і decode** – Інференс LLM складається з двох фаз: *prefill*, яка паралельно обробляє всі токени промпту, щоб ініціалізувати KV-кеш, та *decode*, яка авторегресійно генерує нові токени по одному.

❒ **Спекулятивне декодування** – *Спекулятивне декодування* — метод, що пришвидшує генерування: менша чорнова модель пропонує кілька токенів, які цільова модель потім перевіряє паралельно.

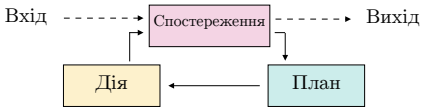
Примітка: метод пришвидшує декодування, бо пропонувати токени малою моделлю дешево, а цільова модель може перевірити кілька запропонованих токенів за один прямий прохід замість генерувати їх по одному.

7 ШІ-агенти

7.1 Огляд

❒ **Визначення** – *ШІ-агент* — система, що автономно досягає цілей і виконує завдання від імені користувача. Для цього вона може використовувати різні ланцюжки викликів LLM.

❒ **ReAct** – *Reason + Act* (ReAct) — фреймворк, що дає змогу використовувати кілька ланцюжків викликів LLM для виконання складних задач:



Цей фреймворк складається з таких кроків:

- *Спостереження*: підсумувати попередні дії та явно зазначити, що відомо наразі.
- *Планування*: описати, які задачі треба виконати і які інструменти викликати.
- *Дія*: виконати дію через API або знайти релевантну інформацію в базі знань.

Примітка: оцінювати агентну систему складно. Втім, це можна робити і на рівні компонентів — через локальні входи й виходи, і на рівні системи — через ланцюжки викликів.

❒ **Обв'язка (harness)** – *Обв'язка* — система навколо моделі, що керує контекстом, інструментами, станом і циклом агента.

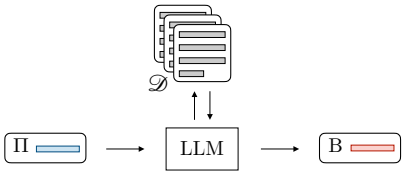
Примітка: приклади — Codex, Claude Code і Pi.

7.2 Пошук (retrieval)

❒ **Ретривер** – *Ретривер* (*retriever*) вибирає з бази знань релевантні документи через *розріджений* (*sparse*) пошук за ключовими словами, *щільний* (*dense*) за ембедингами або їх поєднання.

❒ **Реранкер** – *Реранкер* (*reranker*) переупорядковує знайдені документи за допомогою точнішої моделі релевантності, покращуючи якість контексту, що передається LLM.

❒ **RAG** – *Генерація, доповнена пошуком* (*Retrieval-Augmented Generation*, RAG) — метод, що дає LLM доступ до релевантних зовнішніх знань для відповіді на запитання. Це особливо корисно, коли потрібна інформація, що з'явилася після дати завершення попереднього навчання LLM.



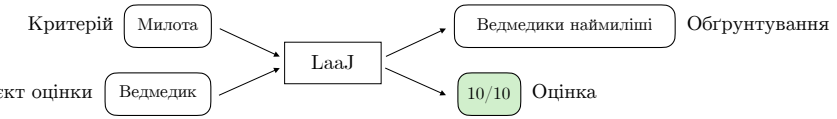
Маючи базу знань $\mathcal{D}$ та запитання, ретривер (**R**etriever) знаходить найрелевантніші документи, потім доповнює (**A**ugments) ними промпт і генерує (**G**enerating) вихід.

Примітка: якість RAG значною мірою залежить від розбиття документів на фрагменти (chunking), якості пошуку та кількості знайдених документів у контексті.

8 Оцінювання

8.1 LLM як суддя

❒ **LLM як суддя** – *LLM як суддя* (*LLM-as-a-Judge*, LaaJ) — метод, що використовує LLM для оцінювання виходів за заданими критеріями. Важливо, що він також може генерувати обґрунтування своєї оцінки, що покращує інтерпретовність.



На відміну від метрик до епохи LLM, як-от *ROUGE* (*Recall-Oriented Understudy for Gisting Evaluation*), LaaJ не потребує еталонного тексту, тож ним зручно оцінювати будь-які задачі. Зокрема, LaaJ демонструє сильну кореляцію з оцінками людей, коли спирається на велику потужну модель (наприклад, GPT-6), оскільки для якісної роботи потрібні здатності до міркування.

Примітка: LaaJ зручний для швидких раундів оцінювання, але важливо стежити за узгодженістю виходів LaaJ з оцінками людей, щоб уникнути розбіжностей.

❒ **Типові зміщення** – Моделі LaaJ можуть мати такі зміщення:

	Позиційне зміщення	**Зміщення багатослівності**	**Зміщення на свою користь**
Проблема	Перевага першій позиції у парних порівняннях	Перевага більш багатослівному вмісту	Перевага виходам, що їх згенерувала сама модель
Рішення	Усереднювати метрику за випадкових позицій	Додати штраф за довжину виходу	Суддя на основі іншої базової моделі

Примітка: наведений перелік зміщень не є вичерпним.

8.2 Оцінювання агентів

❒ **Компонент** – Оцінювання на рівні *компонентів* вимірює якість окремих модулів агента, як-от пошуку, вибору інструментів чи виконання дій.

Примітка: метрики — точність, влучність, повнота, успішність викликів інструментів, затримка.

❒ **Траєкторія** – Оцінювання на рівні *траєкторії* аналізує всю послідовність спостережень, рішень і дій агента.

Примітка: ключові метрики — частка успішно виконаних задач, кількість кроків/викликів інструментів, частка помилок, затримка та вартість.

9 Дифузійні LLM

9.1 Парадигма

❒ **ARM** – *Авторегресійна модель* (*AutoRegressive Model*, ARM) генерує послідовність по одному токену, обумовлюючи кожен новий токен усіма попередньо згенерованими.

❒ **Дифузійна LLM** – *Дифузійна LLM* (dLLM) генерує текст через ітеративний процес знешумлення всієї послідовності — на відміну від ARM, які генерують токени послідовно зліва направо. Два основні типи dLLM:

- *Неперервна дифузія*: знешумлює неперервні ембединги токенів, а потім відображає отримані ембединги на дискретні токени.
- *Дискретна дифузія*: знешумлює токени безпосередньо в дискретному просторі.

Примітка: дискретна дифузія нині є найпоширенішим формулюванням дифузійних LLM, зокрема через масковану дифузію, де токени поступово маскуються та відновлюються.

9.2 Навчання

❒ **Мета** – Модель навчають відновлювати вихідні токени із *зашумленої* послідовності — тобто такої, де частину токенів замасковано або замінено, — за випадково обраного рівня шуму.

❒ **Прямий процес** – *Прямий процес* (*forward process*) поступово зашумлює чистий текст, наприклад, замінюючи дедалі більшу частку токенів на `[MASK]`.

9.3 Інференс

❒ **Стратегія** – Під час інференсу дифузійна LLM починає із зашумленої чи замаскованої послідовності й багаторазово передбачає токени, доки не отримає повну відповідь.

Примітка: дифузійні LLM природно підтримують генерування в довільному порядку та заповнення пропусків (infilling), оскільки генерування не мусить іти строго зліва направо.

❒ **Розклад декодування** – *Розклад декодування* визначає, які передбачені токени приймати, а які повторно маскувати на кожному кроці знешумлення, балансуючи швидкість і якість генерування.